Sparse Small-Cut Responses for Compressed Unknot Hierarchies

Connected graph grammars, budget-preserving composition, and source-addressable terminal certificates

Research continuation prepared for the ProveIt project

Prepared with ChatGPT; proofs and software supplied for review

12 October 2026

Abstract

A large interface does not force an exponential number of states for a fixed small-cut query. For a connected graph with r marked boundary vertices and positive integral edge capacities, the terminal colorings induced by cuts of capacity at most k number at most $2\sum_{j\le k}\binom{2r-3}{j}$ when $r \ge 2$. Two exact monochromatic-edge flags increase the bound by only a factor of four. A terminal spanning-tree skeleton proves the bound without constructing a mimicking graph.

We use this sparsity to evaluate edge-owned graph grammars with arbitrary nesting and fresh-copy reuse. Compatible responses compose exactly under terminal identifications and forgetting. More strongly, indexing by the remaining cut budget bounds the number of candidate pairs by a degree-k, rather than degree-$2k$, polynomial in the interface size. This degree is sharp. At the boundary-pattern threshold $k = 3$, a grammar all of whose pieces are connected admits polynomial-time evaluation in its encoded size, with no logarithmic-width hypothesis and no expansion of binary multiplicities. A second theorem permits disconnected pieces, with an exponential factor only in the number of components meeting the current terminals; arbitrarily many hidden components are harmless for this query.

The evaluator emits complete response certificates, separately checkable witness DAGs, and at most k original cut-edge occurrence addresses. A standard-library implementation passes 37 unit tests, exhaustive checks on 772 connected simple graphs through five vertices, 95,768 terminal-subset projections, 2,500 randomized leaf/grammar cases, and 3,277 complete certificate replays. Paired local measurements include both substantial compression gains and small-input regressions. The result extends the incoming flat four-copy kernel to nested, nonuniform reuse. It is an exact boundary-graph operator, not an unrestricted quasi-polynomial unknot recognizer: the source-faithful grammar producer and full geometric hierarchy accounting remain explicit obligations.

Contents

1 Research position and the next bottleneck **4**
1.1 What is already available 4
1.2 Relation to external results and novelty 4
1.3 The exact scope of the delivery 4

2 Marked cut responses and graph grammar semantics **5**
2.1 Graphs, capacities, and exact flags 5
2.2 Fresh-copy, edge-owned composition 5
2.3 An explicit-leaf encoding contract 6

3 Why small-cut response tables are sparse **6**
3.1 A terminal tree skeleton 6
3.2 Only terminal-bearing components matter 7
3.3 Sharpness and indispensable hypotheses 7

4 Exact composition and contextual replacement **8**

5 Composition preserves the degree of the cut budget **9**
5.1 An implementation rule with a provable effect 9

6 Evaluation of connected and component-bounded grammars **10**
6.1 Compiling an explicit leaf without all vertex colorings 10
6.2 An unconditional local evaluation theorem 11
6.3 Bit sizes, powers, and large hidden multiplicities 11
6.4 Connected assembly can sometimes be arranged 12

7 Graph metadata without expansion **12**

8 Complete certificates and succinct counterexamples **13**
8.1 Soundness and completeness are different checks 13
8.2 At most three original edge addresses at the terminal threshold 13
8.3 Independent complete-table replay 14

9 The boundary-pattern terminal query **14**
9.1 Why the two edge flags are sufficient at threshold three 14
9.2 Ordinary minimum cuts and geometric provenance 15

10 Reproducible validation and measured performance **16**
10.1 Correctness evidence 16
10.2 Stress tests that exceed explicit storage 16
10.3 Paired complete-query measurements 17
10.4 What was not tested 18

11 Integration contracts and corrections to tempting shortcuts **18**
11.1 An additive integration path 18
11.2 Precise corrections and non-corrections 18

12 Further research questions **19**
12.1 A source-complete grammar producer 19
12.2 Controlling terminal-component complexity 19
12.3 Combining four-copy reduction and sparse evaluation 19
12.4 New terminals without expansion 20
12.5 From cut-edge addresses to geometric discs 20
12.6 Factorized responses when C is large 20
12.7 Other finite local witness algebras 20
12.8 Faster complete replay and incremental updates 20
12.9 Sharp constants and structural subclasses 20
12.10 End-to-end hierarchy accounting 21

13 Conclusion **21**

A Source audit and claim-status ledger **21**
A.1 Targeted sources actually inspected 21

B Reproduction and file inventory **22**

C A minimal exact-flag example **22**

References **23**

1 Research position and the next bottleneck

1.1 What is already available

The maintained ProveIt parallelity article at revision `1c0383bd8fdb3865e97f9fbc90171f770106140e` already constructs all-dimensional source parallelity data. It retains the base surfaces, fibre twisting, source charts, and an inherited complementary core with at most $30T$ handles and a conservative $66T$ port bound for T source tetrahedra [1]. We do not re-propose that construction. The remaining distinction between a bounded inherited core and an admissible patterned hierarchy is important: the former alone does not authorize the local hypotheses of the latter.

Three incoming contributions sharpen the present target. The corner-aware report retains the endpoint identifications needed for exact boundary surgery [3]. The interface-homology report separates a small interacting presentation from binary surface multiplicities, and treats ball terminals only with appropriate geometric provenance [4]. The four-copy report removes large connected serial repetitions from a marked small-cut query by a constant-copy replacement, but its width-independent explicit kernel is stated for a flat descriptor [2]. That report asks about nested reuse and marked threshold summaries without exponential interface state.

Here the object being compressed is a *function*: which boundary colorings admit a small cut, at what minimum cost, and with which monochromatic-edge witnesses? The function is sparse whenever the represented piece is connected. This bypasses the need to construct a smaller replacement graph and survives arbitrary DAG reuse under a precise gluing contract.

1.2 Relation to external results and novelty

Lackenby's July 2026 paper gives a hierarchy algorithm and a ball-pattern terminal criterion; its operation costs and hierarchy accounting must be treated separately [5]. Our graph operator supplies one local cost term. It does not replace the surrounding topological algorithm.

Preserving small terminal cuts is an established objective. Chalermsook et al. construct connectivity-threshold mimicking networks, and Liu gives a polynomial-time sparsification result [6, 7]. Our output is not such a network. It retains two *exact* witness flags and supports occurrence-sensitive grammar composition. We neither improve those graph-size bounds nor claim to originate terminal cut compression.

The contributions developed here are the marked sparse-response calculus, the degree-preserving budget-pair bound, the connected-grammar evaluation theorem, the terminal-component refinement, and their certificate implementation. The spanning-tree counting idea is elementary. We supply complete proofs and sharp examples, but do not claim bibliographic priority for every equivalent formulation of these statements. The novelty claim relative to the inspected repository is narrower: nested connected grammar evaluation is not supplied by the incoming flat four-copy operator.

1.3 The exact scope of the delivery

The implementation takes a graph grammar, not a knot diagram or triangulation. It returns graph responses and proof objects, never an unknot verdict. No native source-chart adapter, spherical embedding certificate, or geometric disc transporter is implemented here. The archive includes reproducible graph-level evidence and an additive integration plan. The native recognizer's regression suite was not executed in this environment; the source audit in Appendix A specifies what was actually read.

2 Marked cut responses and graph grammar semantics

2.1 Graphs, capacities, and exact flags

All graphs are finite undirected multigraphs. Loops and parallel edges are allowed. An edge e has a positive integral capacity $w(e)$. The terminal-pattern application uses unit capacities. A piece is a graph G together with an ordered tuple $T = (t_1, \dots, t_r)$ of distinct terminal vertices. Other vertices are private.

For $x : V(G) \to \{0, 1\}$ put

$$q_G(x) = \sum_{uv \in E(G)} w(uv)[x(u) \neq x(v)], \qquad F_G(x) = \{a \in \{0, 1\} : \exists uv \in E(G),\ x(u) = x(v) = a\}.$$

A loop contributes no cut cost and contributes a flag in its vertex's color. The two flags are *exact*: the absence of color a from $F_G(x)$ asserts that there is no monochromatic edge of that color.

Definition 2.1 (Truncated marked response)**.** For a nonnegative integer k, a terminal coloring $\chi \in \{0, 1\}^r$, and $F \subseteq \{0, 1\}$ define

$$\mathcal{R}^k_{G,T}(\chi, F) = \min\{q_G(x) : x|_T = \chi,\ F_G(x) = F\},$$

with all values exceeding k, including an empty minimum, represented by a single absent value ∞_k. The stored table consists only of finite entries. A *marked small cut* at a closed root is a finite entry with $F = \{0, 1\}$.

The software uses a bit mask for χ and the integer $[0 \in F] + 2[1 \in F]$ for F. One minimum witness is retained for each finite key. Different flag sets with the same χ must not be merged.

2.2 Fresh-copy, edge-owned composition

A grammar is a finite acyclic list of rules. A child rule precedes its parent. There are three operations.

An *explicit leaf* lists a finite graph, its capacities, and its terminal tuple. A *binary join* takes fresh disjoint copies of two children, identifies specified child terminal vertices into quotient classes, and retains an ordered subset of the quotient classes as the parent's terminals. A *unary map* performs identifications and terminal forgetting within one child. Multiple terminals of a child may be identified, so a non-loop edge can become a loop. Parent terminal classes are distinct. Private vertices are never identified or promoted by these operations.

Every edge belongs to exactly one child occurrence. Edges are neither identified nor deleted. Two references to the same rule create distinct copies of its private vertices and edges. Thus a DAG is a shared description of fresh occurrences, not a request to alias the underlying graph object. These semantics are essential for cost addition and witness lifting.

Example 2.2 (Serial composition and binary powers)**.** A width-w tile has $2w$ distinct terminals, arranged as an input tuple and an output tuple. Its serial join with another tile identifies output with input coordinatewise and retains the two exterior tuples. Repeated squaring and binary accumulation express a positive power $A^{\circ N}$ using $O(\log(N + 1))$ join rules. A closing unary map identifies the exterior input and output tuples and forgets them. No copies of the represented graph are expanded by the compiler.

More general joins need not be serial or planar. Their graph responses remain valid; applying them to geometry requires a separate faithful importer.

2.3 An explicit-leaf encoding contract

A leaf is explicit, not a second succinct graph language. Its size includes its listed vertices and edges. To make this convention true for the JSON format, the validator requires each vertex to appear in an edge, in the terminal list, or in a list of private isolated vertices. A singleton isolated leaf is the sole harmless implicit exception. Hence

$$|V(\text{leaf})| \le 2|E(\text{leaf})| + |T| + |\text{listed private isolated vertices}| + 1.$$

A binary integer declaring 2^{128} unlisted isolated vertices is rejected before allocating an array of that length. Large repeated isolated components are instead represented by a grammar. This condition closes a potential encoding loophole in any claim of polynomial time in serialized input size.

3 Why small-cut response tables are sparse

3.1 A terminal tree skeleton

Lemma 3.1 (Terminal skeleton)**.** *Let G be connected and let T contain $r \ge 2$ terminals. There is a tree S containing all terminals and at most $2r-3$ edges such that every coloring of G with cut capacity at most k induces at most k bichromatic edges of S. A root color and the set of bichromatic skeleton edges determine the induced terminal coloring.*

Proof. Choose a spanning tree of G, take its minimal subtree containing T, and suppress each nonterminal vertex of degree two. The resulting tree is S. Every leaf of S is a terminal, and every nonterminal vertex has degree at least three. If b is the number of nonterminal vertices, the tree degree sum gives $b \le r-2$, so S has at most $r+b-1 \le 2r-3$ edges.

An edge of S corresponds to a path in the original spanning tree. Distinct skeleton edges have edge-disjoint paths. If the endpoint colors of a skeleton edge differ, its path contains an odd, and therefore positive, number of bichromatic original edges. Positive integral capacities imply that a coloring of capacity at most k has at most k bichromatic graph edges. Consequently at most k skeleton edges have differently colored endpoints. Starting from a chosen skeleton root, flip the color precisely when crossing a specified bichromatic edge. This uniquely recovers all skeleton vertex colors, including the terminals. □

The skeleton is an existence and counting device. The evaluator need not discover or construct a spanning tree of the exponentially large represented graph. It obtains only realizable responses by composing already-computed child tables.

Theorem 3.2 (Connected sparse response)**.** *Let G be connected with r terminals. The number of terminal colorings in the finite support of $\mathcal{R}^k_{G,T}$ is at most*

$$Q_k(r) = \begin{cases} 1, & r = 0, \\ 2, & r = 1, \\ \min\left\{2^r,\ 2\displaystyle\sum_{j=0}^{\min\{2r-3,k\}} \binom{2r-3}{j}\right\}, & r \ge 2. \end{cases}$$

There are at most $4Q_k(r)$ finite marked entries. In particular, for fixed k there are $O_k((r+1)^k)$ entries, and for $k=3$ there are $O((r+1)^3)$ entries.

Proof. For $r \geq 2$, apply Lemma 3.1. There are two choices of root color and at most $\sum_{j \leq k} \binom{|E(S)|}{j}$ choices of the bichromatic skeleton edge set. There cannot be more than 2^r distinct terminal colorings. The cases $r = 0$ and $r = 1$ are immediate. Each terminal coloring has at most four exact flag sets. The asymptotic bound follows from the fixed-degree binomial sum; the constant cases can be absorbed into $(r+1)^k$, including $k = 0$. □

3.2 Only terminal-bearing components matter

For an arbitrary graph let c_T be the number of connected components meeting T. Their terminal counts are $r_1, \ldots, r_{c_T}$. Put

$$d_T = \sum_{i=1}^{c_T} d(r_i), \qquad d(1) = 0, \quad d(r) = 2r - 3 \ (r \geq 2).$$

Components disjoint from T are called hidden. The term refers only to this interface, not to missing geometric information.

Theorem 3.3 (Terminal-component sparse response)**.** *For any graph with positive integral capacities,*

$$|\operatorname{supp} \mathcal{R}^k_{G,T}| \leq 4 \min \left\{ 2^r, \ 2^{c_T} \sum_{j=0}^{\min\{d_T, k\}} \binom{d_T}{j} \right\}. \tag{1}$$

The bound is independent of the number and size of hidden components.

Proof. Choose a terminal skeleton in each terminal-bearing component with at least two terminals. In a one-terminal component retain just that terminal as a root. The skeletons have disjoint edge sets. A coloring of total cost at most k uses at most k bichromatic skeleton edges across their entire disjoint union. Its terminal colors are determined by one root bit per terminal-bearing component and the set of those edges. This proves the boundary-color bound. Hidden components do not supply any terminal color; their contribution can only affect the minimum cost and the global flag set. There are still only four flag sets for any fixed terminal coloring. □

Corollary 3.4 (Uniform convenient bound)**.** *With r terminals and $c_T = c$, the number of finite entries is at most*

$$4 \min \left\{ 2^r, \ 2^c \sum_{j=0}^{\min\{2r, k\}} \binom{2r}{j} \right\}.$$

This is the conservative assertion checked by the reference producer.

3.3 Sharpness and indispensable hypotheses

Proposition 3.5 (The polynomial degree and component factor are necessary)**.** *For each fixed $k \geq 1$, connected graphs with r terminals can have $\Omega_k(r^k)$ finite terminal colorings. At $k = 0$, graphs with c terminal-bearing components can have 2^c finite terminal colorings.*

Proof. Take a star with r terminal leaves and an unmarked center. Color its center zero and choose any k leaves to color one. For $r \geq k$ these give $\binom{r}{k}$ distinct terminal colorings of cut capacity k. For the second assertion take c isolated terminal vertices. Every boundary coloring has zero cost and no edge flags. □

The lower bound already holds for planar connected graphs. It is a lower bound on explicitly listing the response states, not on every possible compressed representation of the response function.

Positive capacities are essential. A connected star of zero-capacity edges has all 2^r boundary colorings at cost zero. Permitting arbitrarily small positive rational capacities without a lower bound gives the same obstruction at a fixed numerical threshold. For capacities bounded below by $\varepsilon > 0$, the counting argument uses $\lfloor k/\varepsilon \rfloor$ in place of k. Negative capacities also invalidate the later pruning of high-cost child states.

Finally, connectedness of the root alone is not enough to give small tables throughout an arbitrary parse. A rule may expose many disconnected terminal components that only become connected later. The maximum c_T over *all* rules, not just the root, is the relevant parameter.

4 Exact composition and contextual replacement

For a join, let $\pi_A : T_A \to Q$ and $\pi_B : T_B \to Q$ be the maps to quotient classes, and let $U \subseteq Q$ be the ordered parent terminal set. Two child states are compatible when their terminal colorings induce a single coloring η of Q. Compatibility includes repeated labels within a single child map. For a compatible pair with costs a, b and flags F_A, F_B, propose the parent state

$$(\eta|_U, F_A \cup F_B) \quad \text{at cost } a + b. \tag{2}$$

Discard it if $a + b > k$ and minimize over all remaining proposals for the same key. A unary map uses the same rule with one child and no cost addition.

Theorem 4.1 (Exact gluing and forgetting)**.** *Rule* (2)*, and its unary version, computes the complete truncated marked response of the graph specified by the grammar operation.*

Proof. Any coloring of the quotient graph pulls back to child colorings whose terminal colors agree on every identified vertex. Every parent edge is one child-occurrence edge, so cut costs add and exact flags combine by union. If the parent cost is at most k, both child costs are at most k because capacities are nonnegative. Replace either restricted child coloring by a minimum witness with the same terminal colors and exact flags. This preserves compatibility, cannot increase the cost, and preserves the proposed parent flag set. Hence the algorithm's minimum is no larger than any valid parent realization.

Conversely, take two compatible finite child response entries and choose their minimum witnesses. Agreement on quotient classes lets the two colorings descend to a well-defined coloring of the joined graph. Private vertices impose no additional identification condition. Edge ownership gives cost exactly $a + b$, even when identified endpoints turn an edge into a loop. Its flag set is precisely $F_A \cup F_B$. Every retained proposal is therefore realized. The two inequalities establish equality of the minima. The argument for unary identification and forgetting is the same without the second child. □

Corollary 4.2 (Contextual response equivalence)**.** *Two pieces with the same ordered terminal interface and equal marked responses can replace one another in any grammar context made of the permitted operations without changing any response value up to k at the root.*

Proof. Theorem 4.1 says that the parent response depends only on the child responses and the stated interface maps. Apply this observation successively to every ancestor of each replaced occurrence in the acyclic grammar. □

This is an equivalence for the specified query. It does not preserve graph isomorphism, homology, genus, an embedding, or a surrounding 3-manifold. Forgetting a terminal is safe only when subsequent contexts will not need to address that vertex. A new marked point must be inserted in an authenticated source description before it can be used as a terminal.

5 Composition preserves the degree of the cut budget

Sparsity alone would suggest trying $O(r^{2k})$ pairs of child states. That is unnecessary. A pair whose minimum costs exceed the shared budget cannot contribute, and there are far fewer pairs that can contribute.

Theorem 5.1 (Budget-pair bound)**.** *For two pieces A, B, let c_A, c_B and d_A, d_B be their terminal-component and skeleton parameters. The number of pairs of finite exact-flag response entries whose minimum costs sum to at most k is at most*

$$P_k(A,B) = 16\,2^{c_A+c_B} \sum_{j=0}^{\min\{d_A+d_B,k\}} \binom{d_A+d_B}{j}. \tag{3}$$

This counts pairs before testing interface compatibility. In particular, when both pieces are connected, the number is $O_k((r_A+r_B+1)^k)$, not $O_k((r_A+r_B+1)^{2k})$.

Proof. Fix terminal skeletons for both pieces. For each finite response entry choose one minimum-cost realizing coloring, with a deterministic tie-breaking rule if necessary. Encode the entry by its terminal-component root colors, the bichromatic skeleton edge set in that witness, and its exact flag set. This encoding is injective on response entries: the root colors and edge set determine the terminal coloring, and the flag set completes the response key. The cost of the chosen witness is the table's minimum and is not an additional choice.

If the entry has cost a, its bichromatic skeleton set has at most a elements. Thus a pair of entries with costs $a+b \le k$ is encoded injectively by $c_A + c_B$ root bits, two flag sets, and a subset of the disjoint union of the two skeleton edge sets having at most k elements. There are at most $2^{c_A+c_B}$ root-bit choices, $4^2 = 16$ flag choices, and the binomial-sum number of edge subsets in (3). Compatibility can only discard pairs, so the same bound applies to compatible candidates. □

Proposition 5.2 (Sharp degree for a budget-indexed join)**.** *Let A and B each be a unit-edge star with a leaves, with every vertex terminal. Join their centers and retain every vertex as a terminal. For $a \ge k$, the number of compatible child pairs of total cost at most k, and the number of parent response entries, are both exactly*

$$2\sum_{j=0}^{k} \binom{2a}{j}. \tag{4}$$

Proof. All vertices are terminals, so a response key fixes the entire coloring and therefore fixes its flags and cost. A compatible pair is determined by the common center color and the subset of the $2a$ leaves whose colors differ from that center. Each such leaf contributes one to the cost. Conversely every choice of at most k leaves gives one compatible pair and one distinct parent coloring. This proves (4) and the degree-k lower bound. □

5.1 An implementation rule with a provable effect

Group the right-child entries first by minimum cost and then by their colors on the shared quotient classes. Reject states inconsistent with self-identifications. For each valid left entry of cost a, inspect only right buckets of costs $0, \dots, k-a$ and the matching shared color. For every candidate, compute the parent coloring and flag union, and update its minimum.

```
for right state z:
    if z respects repeated right labels:
        bucket[cost(z), shared_color(z)].append(z)
for left state y:
    if y does not respect repeated left labels: continue
    for b in 0 .. k-cost(y):
        for z in bucket[b, shared_color(y)]:
            key = (parent_color(y,z), flags(y) OR flags(z))
            keep the minimum cost(y)+cost(z) for key
```

The total number of innermost iterations is bounded by Theorem 5.1. There is also the cost of indexing entries, mapping their interfaces, and probing at most $k+1$ cost buckets per left entry. The reference program implements this rule. The control option `cost_indexed=False` keeps the same shared-color filtering but tries all entries in a matching shared-color bucket before checking the sum; it is used only for paired ablation.

A deterministic implementation can sort records lexicographically and use comparison-based dictionaries. If U is the combined child support size, P the number of feasible pairs, R the combined interface length, and B a bound on the relevant integer bit lengths, a conservative bound is

$$O\Big(((k+1)U+P)(R+B+1)^2\log(U+P+2)\Big) \tag{5}$$

bit operations, after fixed graph metadata overhead is absorbed. This deliberately loose polynomial factor allows explicit bit strings and comparisons. The precise combinatorial gain is the degree-k pair count. Python dictionaries and large-integer bit masks are convenient reference choices; no unit-cost big-integer or adversarial hashing assumption is needed for the overall polynomial classification, and (5) is not asserted to be a literal instruction count for CPython.

6 Evaluation of connected and component-bounded grammars

6.1 Compiling an explicit leaf without all vertex colorings

Lemma 6.1 (Leaf compilation)**.** *For a connected explicit leaf with v vertices and m edges, its complete response is found by considering at most*

$$2\sum_{j=0}^{\min\{k,v-1\}}\binom{v-1}{j}$$

colorings. Each candidate can be generated and evaluated in polynomial bit time in the explicit leaf size and capacity lengths.

Proof. Choose a rooted spanning tree. A coloring is uniquely specified by its root color and the tree edges across which the color changes. Any coloring of graph cut capacity at most k changes on at most k tree edges. Enumerate all subsets of at most k tree edges and both root colors; traverse the tree to generate each coloring and scan all graph edges to compute its exact cost and flags. Keep only costs at most k and minimize for each response key. Every retained coloring is valid, and every low-cost coloring is enumerated. □

The producer implements a subtree-XOR version: precompute a bit mask for each rooted subtree and flip those masks for the selected tree edges. A disconnected leaf is handled component by component, merging its local responses under the same shared total budget. Terminal coordinates for different components are disjoint. Hidden components collapse into at most four exact-flag entries instead of generating independent root-bit choices in the final boundary table.

6.2 An unconditional local evaluation theorem

Let s be the number of rules, R the largest terminal count of a rule, C the largest number of terminal-bearing components of a rule, and L the total explicit leaf size. Let B include all input capacity lengths, indices, and the binary sizes of graph-count metadata. By the explicit-leaf contract, L is polynomially bounded by the serialized grammar size.

Theorem 6.2 (Sparse grammar evaluation)**.** *For a valid edge-owned grammar with positive integral capacities, all truncated marked responses can be computed exactly, with witnesses, in deterministic time of the form*

$$\mathrm{poly}(s, L, R, B, k)\, \exp\big(O(C + k \log(L + R + 2))\big). \tag{6}$$

For every fixed k, if all represented pieces are connected, the time and output size are polynomial in the encoded grammar size. In particular, the $k = 3$ terminal query has a polynomial-time evaluator for connected grammars with arbitrary interface width and arbitrary nesting.

Proof. Process rules in topological order. Compile each explicit leaf by Lemma 6.1; for disconnected leaves, merge component tables with budget indexing. Intermediate terminal-bearing component counts do not exceed the leaf's final count. The numbers of enumerated tree-switch sets are bounded by $(L+1)^{O(k)}$, and every leaf table obeys Theorem 3.3.

At a unary map, scan the child support, reject inconsistent identifications, project to the retained terminals, and minimize. At a join, use Theorem 4.1 for correctness and Theorem 5.1 for the candidate count. Since $d_A + d_B \le 4R$ and $c_A + c_B \le 2C$, both the table sizes and the cost-feasible pair counts have the form $2^{O(C)}(R+1)^{O(k)}$. Comparison-based indexing and arithmetic add only polynomial factors as in (5). Summing over the s rules proves (6). Minimum witnesses are one backpointer record per stored state and therefore have the same output-size classification.

If all represented pieces are connected, every c_T is at most one, including $c_T = 0$ for a closed piece. A fixed k makes every remaining exponent constant. Nested reuse changes the number of graph occurrences but not the number of grammar rules processed. □

Corollary 6.3 (A local quasi-polynomial regime)**.** *Suppose s, L, R, B are polynomially bounded in a parameter $n \ge 2$ and*

$$C + k \log(L + R + 2) = O((\log n)^2).$$

Then the supplied grammar query is computable in $n^{O(\log n)}$ time. At the fixed threshold $k = 3$, it is enough that $C = O((\log n)^2)$; the terminal count R may be polynomial in n.

The theorem is unconditional for its graph-grammar input. The corollary is conditional only on stated input-parameter bounds. Neither asserts that every transported boundary pattern of every unknot hierarchy admits such a grammar.

6.3 Bit sizes, powers, and large hidden multiplicities

A rule's expanded vertex and edge counts are at most 2^s times the largest explicit leaf size: a join adds two child counts before vertex identifications decrease the vertex count. Thus graph-count bit lengths are $O(s + \log(L+1))$. Costs retained in responses lie in $\{0, \dots, k\}$; reading a large capacity still costs its actual bit length. No loop in the evaluator runs for the numerical expanded multiplicity.

For instance, starting with a closed single-edge graph and repeatedly taking the disjoint union of two fresh copies yields 2^h hidden components after h joins. Every interface is empty and has at

most four response keys. This family verifies why a bound in the total number of components would be unnecessarily pessimistic. Conversely, c isolated *terminal* vertices really do require 2^c zero-cost boundary states in this representation.

The numeric parameter k is not silently charged by $\log k$. The leaf enumeration and the explicit budget-bucket loop depend on k. The terminal application fixes it at three; the parameterized bound (6) states the dependence for other uses.

6.4 Connected assembly can sometimes be arranged

Lemma 6.4 (Connected overlap assembly)**.** *Suppose finitely many connected edge-owned pieces are to be joined only at listed terminals. If their overlap graph is connected, the union has a binary assembly in which every intermediate represented piece is connected, provided every vertex needed for a later attachment remains terminal until that attachment occurs.*

Proof. Take a spanning tree of the overlap graph and order its vertices so that each piece after the first has a neighbor earlier in the order. Join pieces in that order, keeping the listed future attachment vertices. At each step the new connected piece intersects the connected previous union in at least one terminal class, so the union stays connected. All prescribed additional identifications can be made when their endpoints are present; a quotient of a connected graph remains connected. □

The lemma is a parse-construction tool, not a bound on the size of a native grammar. Retaining the necessary interface can enlarge R, and reorganizing a DAG can destroy useful sharing. Those costs must be recorded by a producer.

7 Graph metadata without expansion

A terminal importer must know more than the response function. The implementation independently tracks the expanded vertex and edge counts, the partition of terminals by graph component, the number of hidden components, terminal degrees saturated at four, and whether every private vertex has degree three. This certifies abstract cubic graph structure without inspecting every expanded vertex.

Proposition 7.1 (Compositional connectivity and cubic metadata)**.** *The stated metadata can be computed exactly from child metadata and terminal quotient maps in polynomial time per rule, with graph counts stored in binary.*

Proof. Let Q be the set of quotient classes in a join. On Q, identify all classes containing terminals from the same connected component of either child. The resulting partition is exactly the component relation among quotient vertices: each equivalence step is a path in a child component, and any path in the joined graph decomposes into such child paths. Child hidden components cannot meet the interface and remain separate.

The new hidden-component count is the sum of child hidden counts plus the number of classes of this connectivity partition that contain no retained parent terminal. Restriction to retained terminals gives the new boundary partition. Edge counts add. If the children have v_A, v_B vertices and r_A, r_B terminals, then

$$v_{A\vee B} = v_A + v_B - r_A - r_B + |Q|.$$

Private vertices were not identified, and exactly $r_A + r_B$ distinct terminal occurrences were replaced by $|Q|$ vertices.

The degree of a quotient vertex is the sum of degrees of its incident child terminal occurrences. If an edge becomes a loop, its two endpoint incidences both remain in this sum, as required. Saturation at four commutes with adding nonnegative degrees and suffices to distinguish degree three. Every newly forgotten quotient vertex must have degree three, and the previously private vertices must already have passed that check. This gives the private-cubic flag. Unary maps follow the same argument with one child. □

At a terminal-free root, the private-cubic flag checks that every graph vertex has degree three. It says nothing about spherical embeddability, the provenance of the graph, or whether its carrier is the boundary of a ball. The metadata certificate is therefore useful but cannot be mistaken for a geometric certificate.

8 Complete certificates and succinct counterexamples

8.1 Soundness and completeness are different checks

The producer stores a minimum witness for every finite state. At a leaf this is a coloring bit mask. At an internal rule it is the list of child response keys whose compatible minimum witnesses realize the parent entry.

A *soundness-only witness* verifies one selected root response. A *complete response certificate* verifies that no finite states were omitted and every stored minimum is correct. The second is required to certify absence of a marked small cut. Checking one convenient coloring cannot establish such absence.

Theorem 8.1 (Witness DAG bound)**.** *A finite root response admits a witness DAG containing at most one record per reachable pair (grammar rule, response key). Its record count is at most the total number of stored response entries. It can be checked without expanding the represented graph.*

Proof. Start with the selected root key and recursively follow its stored child keys. Merge requests for the same rule and key. The rule index strictly decreases along every edge, so this is an acyclic graph. A leaf record gives an actual coloring; checking its terminal colors, flags, and cost is explicit. An internal record gives previously checked child colorings that agree on all quotient classes and add to the claimed cost and flags by Theorem 4.1. Induction proves that each record is realized, and hence so is the root. Fresh occurrences can reuse the same record as a recipe; they are not identified as graph occurrences. □

8.2 At most three original edge addresses at the terminal threshold

An original edge occurrence is named by a root-to-leaf sequence of child choices followed by a leaf edge index. The path distinguishes two occurrences of the same rule. Unary steps have a single child choice.

Theorem 8.2 (Source-addressable cut and mark witnesses)**.** *From a witness of cost at most k, one can extract all cut-edge occurrence addresses, at most k of them, and one monochromatic-edge address for each present flag. The total path length is $O((k+2)s)$. Extraction and replay are polynomial in the stored witness, the input bit sizes, and this output length.*

Proof. Skip every witness branch of cost zero. At a positive-cost join, child costs sum to the parent cost, and each positive-cost child has cost at least one. At any depth there are therefore at most k active occurrence branches. The rule depth is at most s, so at most ks positive-cost

occurrence visits are needed. At a reached leaf, scan its explicit edge list and output exactly the bichromatic edges. Their positive integral capacities sum to the root cost; hence there are at most k addresses.

For a color flag, descend through any child whose flag set contains that color. The OR rule ensures such a child exists. At a leaf, scan for a monochromatic edge of the requested color. There are at most two such searches and each has at most s grammar steps. The resulting occurrence paths are unambiguous by fresh-copy edge ownership. Replay follows the named paths through the checked witness, verifies the leaf edges, verifies that cut addresses are distinct, and checks that their capacities sum to the root witness cost. Since all cut capacities are positive, equality of this sum establishes that no cut edge was omitted. □

These are original addresses in the *grammar*. Calling them addresses in a triangulation or a transported surface would additionally require the missing source adapter. A small abstract cut does not by itself supply an embedded noose or a properly embedded violating disc.

8.3 Independent complete-table replay

The separate `checker.py` imports no producer routines. Its leaf completeness algorithm enumerates sets of at most k removed edges, skips sets whose total capacity exceeds k, and colors the components of the remaining graph. Removing at most k edges from a connected component produces at most $k + 1$ components. Every low-cost coloring is encountered when its exact cut-edge set is removed; all generated candidates are evaluated against the original edges. Disconnected leaves are merged componentwise.

At internal nodes, the checker uses literal quotient-class color dictionaries and explicitly recomputes response minima from the verified child tables. It separately recomputes the metadata. A SHA-256 digest binds the certificate to the exact serialized grammar, but is not a substitute for the structural checks. The checker deliberately tries the full child-table Cartesian product. Its replay is therefore polynomial for fixed k and bounded terminal-component count, but it does *not* implement the producer's sharper budget-pair enumeration bound. Its timings are reported separately.

The test suite rejects omitted states, forged costs and leaf witnesses, changed sources, forged metadata, duplicate cut addresses, and confusion between cut-address and mark-address roles. Explicit producer budget exhaustion is reported as an inconclusive query; it cannot become a no-cut certificate. The operation counter measures documented candidate work, not a hard wall-clock or memory limit for hostile inputs.

9 The boundary-pattern terminal query

9.1 Why the two edge flags are sufficient at threshold three

The incoming four-copy report proves the following useful primal formulation of the ball terminal test [2]. We repeat the relevant elementary argument to make the interface contract clear.

Lemma 9.1 (Small cubic cuts)**.** *In a cubic multigraph with unit edges, a cut of size at most three has a monochromatic edge on each shore if and only if each induced shore contains a cycle.*

Proof. Suppose an induced shore is a forest with v vertices and a nonempty components. It has $v - a$ internal edges, so its edge boundary has size

$$3v - 2(v - a) = v + 2a.$$

If the forest contains an edge, then $v \geq 2$ and $a \geq 1$, forcing at least four boundary edges. Thus at threshold three an induced shore containing an edge cannot be a forest. A cycle plainly contains an edge. Apply both observations to the two shores. Loops and parallel-edge cycles are included in the usual multigraph convention. □

The conclusion also holds under minimum degree at least three, with the degree identity replaced by an inequality. At threshold four it fails: two adjacent vertices of K_4 induce an edge, not a cycle, and have a four-edge boundary. This is why raising the query threshold cannot silently reuse the same topological interpretation.

Proposition 9.2 (Certified ball application)**.** *Suppose a supplied grammar faithfully describes the nonempty cubic part P of a boundary pattern embedded in the sphere bounding a certified 3-ball, with no additional vertex-free circle components. Then P is an essential ball pattern if and only if the terminal-free $k = 3$ response has no state with both edge flags.*

Proof. The primal criterion is established in the incoming report [2]. Here is its geometric argument. A simple closed curve transverse to a cubic pattern in at most three points is nonviolating if one side contains only the empty pattern, one arc, or one tripod. A forest lying in a disc, with trivalent interior vertices and at most three boundary leaves, has exactly one of those forms: each nonempty tree with u interior trivalent vertices has $u + 2$ leaves, and two arc components already need four leaves. A violating curve therefore has a graph cycle on each side. Coloring vertices by side gives a cut of at most three edges and both flags.

Conversely, by Lemma 9.1, a marked cut has a cycle on each shore. In a connected graph choose such a cut of smallest cardinality. Both shores are connected: otherwise a cycle-containing component of one shore has a strictly smaller edge boundary, while its complement retains the opposite cycle. A cut with two connected shores is a bond. In a spherical embedding its dual cycle, or the frontier of a suitable regular neighborhood of one shore, is a simple closed curve meeting exactly the bond edges and separating the two cycles. It violates essentiality. If the graph is disconnected, every cubic component contains a cycle; separating components gives both a zero-cost marked cut and a disjoint violating curve. This proves the equivalence. □

9.2 Ordinary minimum cuts and geometric provenance

The distinction between edge connectivity and the marked query is not cosmetic. The graph K_4 and the triangular prism both have ordinary edge connectivity three. In K_4, a three-edge cut isolates a single vertex and does not supply two edge flags. The prism's three matching edges separate its two triangles and do supply both flags. The delivered examples verify these different responses.

The empty pattern and a single vertex-free simple closed curve require separate importer cases. They should not be represented by adding degree-two vertices and then applying a cubic check. Multiple such circles, or a circle together with a cubic component, also require the correct geometric inventory.

Proposition 9.2 assumes a ball, not merely an abstract graph and not merely sphere boundary. The incoming homology report describes a shortcut for genuine cut descendants with certified S^3 embedding provenance, while recording why generic boundary and homology data are insufficient [4]. This contribution does not implement that provenance chain. Nor does it establish the essentiality of an arbitrary initial knot exterior by inspecting its boundary graph. The hierarchy and the interpretation of its terminal certificates remain indispensable.

10 Reproducible validation and measured performance

All results in this section are completed local runs on CPython 3.13.5, Linux x86-64, using only the standard library. They do not measure the native ProveIt recognizer. Raw per-repeat measurements, certificates, witness addresses, platform information, and deterministic seeds are included in `data/`. The final source hashes are listed in the archive manifest.

10.1 Correctness evidence

The 37 unit tests cover exact flags, loops, parallel edges, a 4,097-bit capacity, self-identification, terminal forgetting, disconnected joins, huge hidden component counts, connected powers, nested nonuniform reuse, certificate mutations, budgets, and the explicit-leaf inventory condition. The separate audit has the following coverage.

Validation family	Completed checks
Connected labeled simple graphs, $1 \le \lvert V\rvert \le 5$	772
Graph/threshold pairs, $k = 0, 1, 2, 3$	3,088
All terminal-subset projections of those responses	95,768
Random weighted multigraph leaves	1,000
Random valid grammars compared with literal colorings	1,500
Join candidate bounds checked in those grammars	4,276
Independent complete-certificate replays	3,277
Sharp joined-star instances, $3 \le a \le 12$	10

Table 1: Completed deterministic audit. Counts describe distinct validation tasks, not independent theorem proofs.

The connected-graph counts by vertex size are $1, 1, 4, 38, 728$. The terminal-subset audit projects independently computed full-terminal responses; it is not 95,768 separate geometry extractions. Random grammars use loops, parallel edges, weights from one to four, terminal identifications, forgetting, disjoint joins, and fresh-copy reuse. Of 1,648 generated candidates, 1,500 met the explicit reference caps of at most twelve vertices and sixty edges at every represented rule. Capped candidates are excluded from the comparison count, not reported as passes.

The independent small oracle enumerates all vertex colorings of the literal expanded graph. The complete checker uses edge-deletion enumeration at leaves, a different completeness argument from the producer's tree-switch enumeration. The sharp star audit checks equality with formula (4), not merely an upper bound. None of these finite tests replaces the proofs in Sections 3–8.

10.2 Stress tests that exceed explicit storage

The saved connected ladder with 2^{256} tile copies has 2^{257} vertices and $3 \cdot 2^{256}$ edges. Its grammar has 258 rules, peak support 32, and 7,700 total stored states. Its complete response certificate independently replays and has no marked cut. This is an exact computation on that graph grammar, not an experiment on an equally large knot diagram.

The nonuniform example uses connected tiles A, B and recursive productions

$$A_{h+1} = A_h \circ B_h, \qquad B_{h+1} = B_h \circ A_h,$$

followed by closure. At depth 128 it is represented by 259 rules. The second tile includes an internal K_4 with one edge removed, attached through its two deficient vertices; this supplies a genuine small separating cut. The final graph is cubic and has 2^{130} vertices and $3 \cdot 2^{129}$ edges.

Its marked response, complete certificate, and succinct cut/mark addresses all replay. The claim is nested, nonuniform reuse, not a lower bound on every alternative grammar for this family.

The third large example has 2^{128} hidden single-edge components and no terminals. It has 129 rules and peak support four. The root has a marked cut of cost zero. This demonstrates the terminal-component refinement; it is deliberately not a cubic ball-pattern instance.

10.3 Paired complete-query measurements

Copies N	Vertices	Sparse (ms)	Literal (ms)	Literal/sparse
1	2	0.262	0.121	0.46
2	4	0.417	0.168	0.40
4	8	0.671	0.392	0.58
8	16	1.023	3.602	3.52
16	32	0.986	28.235	28.64
32	64	1.162	429.938	369.85

Table 2: Complete ladder-query medians. Ratios below one favor the literal path. No knot-recognition work is included.

Arms a	Root states	Indexed (ms)	Control (ms)	Time ratio	Work ratio
4	186	1.088	1.072	0.99	1.96
8	1,394	7.765	9.809	1.26	9.15
12	4,650	32.092	62.519	1.95	28.03
16	10,978	90.453	247.086	2.73	64.37

Table 3: Cost-indexed star joins versus the same producer without cost indexing. Time and work ratios are control/indexed. Work counts are logical candidate counts, not machine operations.

Copies	Rules	Total states	Solve (ms)	Complete replay (ms)
2^{8}	10	260	1.476	13.388
2^{32}	34	980	5.926	56.015
2^{128}	130	3,860	24.606	224.357
2^{256}	258	7,700	50.953	470.832
2^{512}	514	15,380	112.289	970.095
2^{1024}	1,026	30,740	289.052	2069.746

Table 4: Binary ladder powers. Solve times are medians of five calls on a supplied grammar; complete replay is one separately timed call, with no claim of median precision. Peak table size is 32 in every row.

The two identical A/A controls had median B/A ratios 0.938 and 0.973. These are empirical noise controls, not confidence intervals. The full raw repeat data are retained in `data/benchmark.json`.

For the ladder experiment, both paths start by constructing the same compact grammar. The compressed path computes every grammar response and serializes the complete root response. The literal path expands the grammar, enumerates all low-cut spanning-tree colorings of the explicit connected graph, evaluates them, and serializes the same response. Thus the comparison does not pit a complete query against one cheap internal operation. The literal comparator

shares the elementary tree-switch principle, while the exhaustive vertex-coloring oracle remains the independent correctness reference.

The star ablation uses identical input grammars and output serialization, changing only whether child candidates are indexed by remaining cut budget. All root responses agree. The operation count is a logical candidate-work counter, not a bytecode count. Its reduction can be much larger than the wall-time gain because output masks, minimum updates, validation, and serialization still have to be performed.

Timing includes equal warm-up counts and alternating A/B order, with seven paired repeats for ladders and five for the star ablation. Garbage collection is requested before each measured call. The raw timings retain fluctuations; identical A/A controls are reported in the data. Differences comparable to those controls are not evidence of a speed improvement. The smallest explicit ladders favor expansion, so automatic dispatch rather than unconditional replacement is appropriate. These measurements establish local operator behavior only: no native source extraction, hierarchy repair, or end-to-end knot recognition was timed.

10.4 What was not tested

No native geometric importer is present. We did not run the maintained `fastunknot` or Rust test suites, did not benchmark a knot corpus, and did not verify the full contents of all incoming binary archives. Network access from the working container did not yield a usable repository checkout. The standalone tests do not inherit the native project's reported test totals. Likewise, complete graph-certificate replay is not a Lean formalization and is not a proof of geometric correctness of a future importer.

11 Integration contracts and corrections to tempting shortcuts

11.1 An additive integration path

The package can first be placed under a new research/report directory, with the source modules isolated from the current recognizer defaults. A production adapter should pass an authenticated graph grammar to the sparse evaluator, replay the returned proof object as appropriate, and attach the graph result to its geometric owner. The root graph response and the owner manifold's terminal status must be separate fields.

A source-facing adapter needs an edge-occurrence ownership map, actual terminal vertex identities, an authenticated spherical embedding or carrier, a current ball certificate, and the status of vertex-free circle components. It must justify that all future attachment points were retained as terminals. The existing corner-aware representation is relevant to that last obligation, but a cyclic corner kernel is not already a proof that the grammar describes the complete transported graph.

The integration note proposes `query.py` as a reproducible reference interface. It does not patch any default recognition stage. The suggested next native experiment times the complete chain from source charts to grammar, replay, terminal query, and source-address lifting. It should compare the four-copy route, the sparse DAG route, and a hybrid on the same source-certified instances.

11.2 Precise corrections and non-corrections

The targeted native parallelity article already preserves source attaching charts and distinguishes bounded inherited geometry from admissible patterned structure. We found no contradiction in

that inspected claim and propose no retraction of it. The following are corrections to unsafe *extensions* of the current work, not accusations that the maintained implementation makes them.

First, a dense table over all 2^r terminal colorings is an unnecessarily pessimistic bound for a connected positive-capacity piece and a fixed small-cut threshold. Theorems 3.2 and 5.1 replace that estimate with sparse degree-k support and candidate counts.

Second, independently truncating child costs and then taking their entire Cartesian product misses a provable gain. The shared budget must be used before enumerating candidates. This changes a degree-$2k$ pair estimate to degree k.

Third, counting all graph components overstates the obstruction. Only components that meet the retained terminals contribute independent boundary root bits. Hidden components still affect flags and minimum costs, but not a new exponential family of boundary colorings.

Fourth, raw cut values or ordinary edge connectivity lose the two monochromatic witnesses needed by the terminal test. Exact flags must be propagated by OR, or a different formally sufficient witness object must be proved.

Fifth, DAG reuse is not vertex or edge aliasing. Treating two references as the same graph occurrence changes both capacities and topology. Identifying graph edges across a join also invalidates the additive-cost proof unless a different ownership convention is supplied.

Finally, a polynomial graph-query kernel is not a polynomial or quasi-polynomial knot recognizer. The missing geometric producer and all hierarchy costs remain in the total. This is a scope boundary, not a reason to discard the local theorem.

12 Further research questions

12.1 A source-complete grammar producer

Can every boundary pattern produced by the maintained affine cut and marked parallelity routines be compiled into an edge-owned grammar of polynomial encoded size? A useful first result should identify a nontrivial class of source normal cuts and prove the importer complete for that class. The output must authenticate edge ownership and every attachment vertex, rather than merely fit a small graph with the same observed scalar statistics.

12.2 Controlling terminal-component complexity

Can a producer ensure $C = O(\log^2 n)$, or preferably constant C, throughout the grammar? Lemma 6.4 helps for explicitly listed connected overlapping pieces. Its effect on interface size and DAG sharing remains to be analyzed. Root connectedness alone is not the desired invariant. A parse optimizer should report a certified component profile at every intermediate rule.

12.3 Combining four-copy reduction and sparse evaluation

The flat four-copy operator can eliminate a connected periodic sleeve with no interface-state enumeration. The present evaluator handles arbitrary connected nested reuse. Can a hybrid choose the smaller certified representation locally, with a proof of near-best cost among a defined family of parses? A replacement must preserve the exact marked response, not only ordinary terminal minimum cuts.

12.4 New terminals without expansion

The current grammar cannot promote a private vertex after it has been forgotten. Can a binary source address split the affected grammar occurrence and introduce a new terminal with polynomial increase in grammar size? Repeated insertions can duplicate ancestor contexts, so a one-insertion lemma is not yet a global bound. This is a concrete connection to the corner-aware boundary work.

12.5 From cut-edge addresses to geometric discs

At threshold three, the graph certificate supplies at most three edge occurrences and two monochromatic witnesses. Can source rotation systems and corner charts construct a compressed noose realizing an appropriate bond, then an embedded violating disc in the certified ball? The construction must also transport that disc through earlier hierarchy steps. An abstract planar duality argument establishes existence but not the encoded source cost of this operation.

12.6 Factorized responses when C is large

The factor 2^C is necessary for an explicit table, but not necessarily for a more structured representation. At zero cost, component colors form a simple equality relation. Can low-cost perturbations of that relation be stored as a small decision diagram, a sum of products, or a bounded-rank object that still supports the exact flag algebra? The disconnected permutation examples in the four-copy report are useful test cases rather than evidence of a general solution.

12.7 Other finite local witness algebras

Let every edge contribute, according to its endpoint colors, an element of a fixed finite commutative monoid M. Replace the two-bit OR aggregate by the product in M. The same proofs replace the factors 4 and 16 in (1) and (3) by $|M|$ and $|M|^2$, respectively: the aggregate is simply an extra finite label in the injection and composes associatively. Which additional hierarchy queries admit such an edge-local witness? Cycle existence itself is not an edge-local OR flag in arbitrary graphs; the special cubic threshold-three lemma is doing genuine work here.

12.8 Faster complete replay and incremental updates

A comparison-based cost-indexed checker could obtain the producer's sharper candidate bound without sharing its implementation. Alternatively, complete table certificates might support locally checkable coverage proofs. Can modifications to a small number of source leaves update only their ancestor rules, retaining certificates for unchanged sub-DAGs? A source digest strategy must distinguish unchanged recipes from changed occurrence bindings.

12.9 Sharp constants and structural subclasses

The exponent k is necessary in general, but the factor arising from terminal tree skeletons is loose on many graph classes. Determine sharper support and pair counts for cubic planar pieces, fixed numbers of boundary intervals, or pieces with prescribed cyclic terminal order. A smaller count must follow from an actual structural theorem; planarity alone cannot eliminate the star lower bound for general-degree graphs.

12.10 End-to-end hierarchy accounting

The local result changes one term in an eventual total-cost proof. What bounds the number and encoded sizes of grammar queries, admissibility repairs, hierarchy resets, source surface discoveries, and disc transports together? A global theorem must include all these operations and their interaction with the current compressed core. The strongest immediate milestone would be a source-certified restricted-input recognition theorem whose grammar and hierarchy bounds are both proved, followed by complete native measurements for that class.

13 Conclusion

At a fixed small-cut threshold, connected pieces have sparse marked boundary responses even when the boundary itself is large. A terminal tree skeleton proves the support bound, while a shared-budget injection proves that composition preserves its polynomial degree. This yields a polynomial-time evaluator for connected graph grammars with arbitrary nested reuse, and a component-parameterized quasi-polynomial regime without a logarithmic interface-width assumption.

The delivered implementation computes these responses exactly and makes both completeness and succinct counterexamples independently replayable. It extends the flat repetition result by changing the retained object from an explicit graph kernel to a sparse compositional function. The remaining path to the user's general quasi-polynomial recognition objective is now more sharply located: produce faithful bounded-complexity grammars from the native geometry, transport their small-cut witnesses geometrically, and include their costs in a proved hierarchy accounting.

A Source audit and claim-status ledger

A.1 Targeted sources actually inspected

The maintained `Topology/UnknotRecognition/README.md` was fetched from the default branch with Git blob `4fb929b5689d0bc20327e241cb775789660c8013`. The parallelity article `Topology/UnknotRecognition/synthesis/parallelity.tex` was read at revision `1c0383bd8fdb3865e97f9fbc90171f770106140e`, with blob `d7293976bea14fce6b4848bf6a2d7a20134c2491`. The incoming directory listing identified the four-copy, corner-aware, interface-homology, marked-parallelity, cancellation, and reductions archives. Only the relevant precursor articles retrieved through the user's file Library were read in detail; the listing is not a content audit of all those archives.

The four-copy and corner-aware TeX sources and the interface-homology report inspected here describe the earlier checkpoint `ed166080d7db14dd7a91d07788996f02947dd92a`. That is their source pin, not a claim that the native repository was frozen at that revision during this work. Public primary-source checks included Lackenby's 2026 paper and the two cited threshold sparsification papers. No repository commit or write was made.

Claim class	Status
Sparse support, composition, budget-pair bound	Proved in this article for the exact stated graph model.
Connected-grammar polynomial evaluation	Proved, including explicit-leaf and bit-size contracts.
General quasi-polynomial regime	Proved conditional on the displayed grammar-parameter bounds.

Claim class	Status
Local software and replay	Implemented; 37 unit tests and the recorded audit completed.
Local speed measurements	Completed and archived; include negative small-input results and A/A controls.
Native source-to-grammar integration	Not implemented or measured here.
General quasi-polynomial unknot recognition	Not established by this contribution.
Novelty priority	No exhaustive priority claim; classical antecedents and repository predecessors identified.

B Reproduction and file inventory

From the package root:

```
PYTHONPATH=src python -m unittest discover -s tests -v
PYTHONPATH=src python experiments/audit.py
PYTHONPATH=src python experiments/benchmark.py
PYTHONPATH=src python src/query.py examples/triangular_prism.json \
    --check --certificate /tmp/prism-certificate.json
cd article && sh build.sh
```

The commands require Python 3.10 or later; the recorded runs used 3.13.5. PDF rebuilding requires a LaTeX installation with the packages named in the source. The benchmark is intentionally a separate command from unit testing.

`src/sparse_cuts.py` is the producer; `src/checker.py` is the independent checker; `src/reference.py` contains the capped expansion and explicit-coloring references; and `src/query.py` is the CLI. Tests, deterministic experiments, JSON examples, complete certificates, witness addresses, raw timing rows, source audit, integration notes, and checksums are included. Python bytecode caches and LaTeX intermediate files are excluded from the deliverable.

C A minimal exact-flag example

A single unit edge with no terminals has three finite response entries at threshold at least one:

$$(\varnothing, \{0\}) \mapsto 0, \qquad (\varnothing, \{1\}) \mapsto 0, \qquad (\varnothing, \varnothing) \mapsto 1.$$

It has no both-flag entry. The disjoint union of two fresh copies does have a both-flag entry of cost zero: color one edge zero and the other one. This is why hidden components cannot simply be deleted, even though they introduce no terminal-color bits. The four-state aggregate is precisely what lets their exponentially large multiplicity remain compressed.

References

[1] ProveIt contributors, *Source parallelity in all handle dimensions and a bounded inherited core*, `Topology/UnknotRecognition/synthesis/parallelity.tex`, revision and blob in Appendix A, 2026. `https://github.com/VladimirReshetnikov/ProveIt`.

[2] ProveIt research continuation prepared with ChatGPT, *Four-Copy Kernels for Compressed Boundary Patterns: Marked cut signatures, sharp repetition bounds, and certified terminal tests for unknot hierarchies*, 12 October 2026. Incoming bundle `ProveIt_Unknot_Four_Copy_Research.zip`; article source `four_copy_patterns.tex`.

[3] ProveIt research continuation, *Corner-Aware Boundary Kernels for Compressed Unknot Hierarchies: Exact port surgery, orbit-count compilation, and interval-bundle consequences*, 12 October 2026. Incoming bundle `ProveIt_Corner_Aware_Boundary_Kernels_2026-10-12.zip`.

[4] ProveIt research continuation prepared with ChatGPT, *Bounded-interface integral homology: Binary surface condensation, twisted attachments, and provenance-aware terminals for compressed unknot hierarchies*, 12 October 2026. Incoming bundle `ProveIt_Unknot_Interface_Homology_2026-10-12.zip`.

[5] M. Lackenby, *Incompressible surfaces, hierarchies and unknot recognition*, arXiv:2607.23350v1, 25 July 2026, especially Proposition 2.6 and Section 9. `https://arxiv.org/abs/2607.23350`.

[6] P. Chalermsook, S. Das, B. Laekhanukit, Y. Kook, Y. P. Liu, R. Peng, M. Sellke, and D. Vaz, *Vertex Sparsification for Edge Connectivity*, Proceedings of SODA 2021, pp. 1206–1225; arXiv:2007.07862. `https://doi.org/10.1137/1.9781611976465.74`.

[7] Y. P. Liu, *Vertex Sparsification for Edge Connectivity in Polynomial Time*, ITCS 2023, LIPIcs 251, 83:1–83:15. `https://doi.org/10.4230/LIPIcs.ITCS.2023.83`.